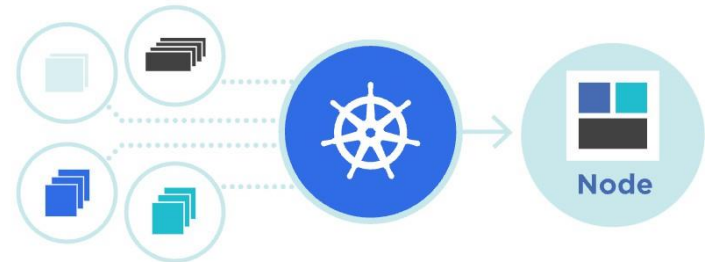

MSML 605

Kubernetes

Introduction

Kubernetes, also known as K8s, is an open-source system for automating deployment, scaling, and management of containerized applications.

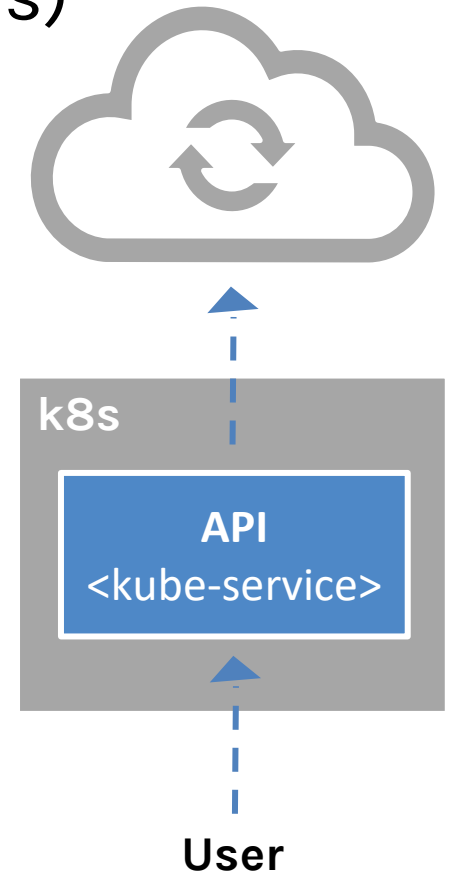
It groups containers that make up an application into logical units for easy management and discovery. Kubernetes builds upon [15 years of experience of running production workloads at Google](#), combined with best-of-breed ideas and practices from the community.



Source: <https://kubernetes.io/>

Why Containers and Container APIs?

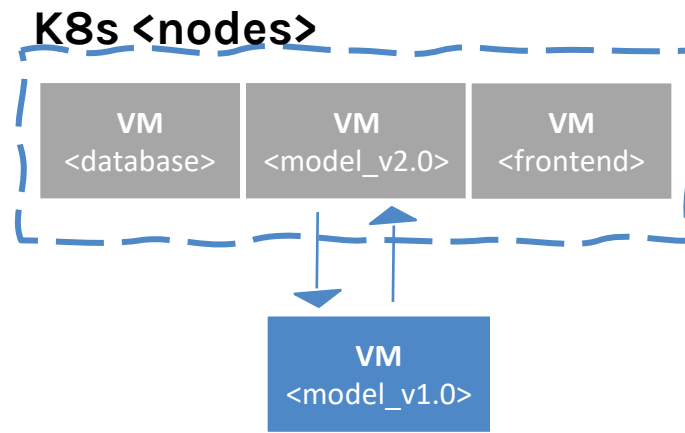
- Velocity
- Scaling (of both software and teams)
- Abstracting the infrastructure
- Efficiency



Velocity

Velocity is enabled by

- **Immutable system:** you can't change running container, but you create a new one and replace it in case of failure (allows for keeping track of the history and load older images)

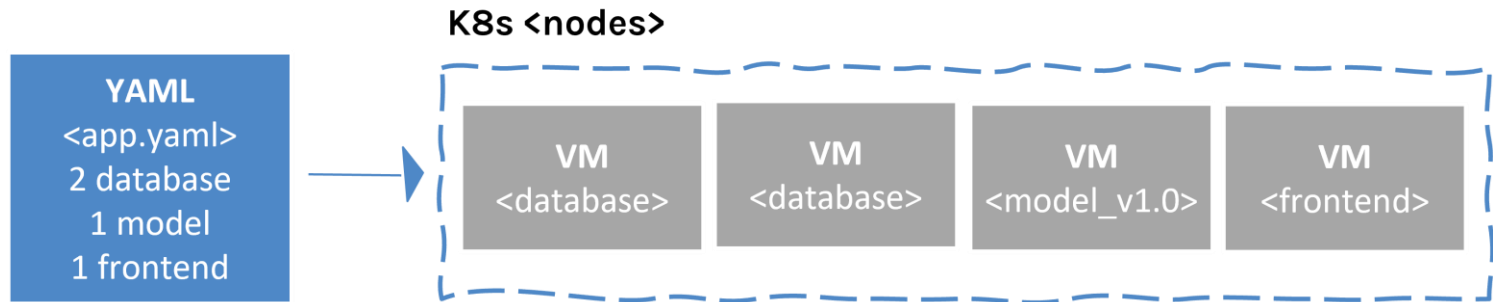


Velocity

Velocity is enabled by

- **Declarative configuration:** you can define the desired state of the system restating the previous declarative state to go back.

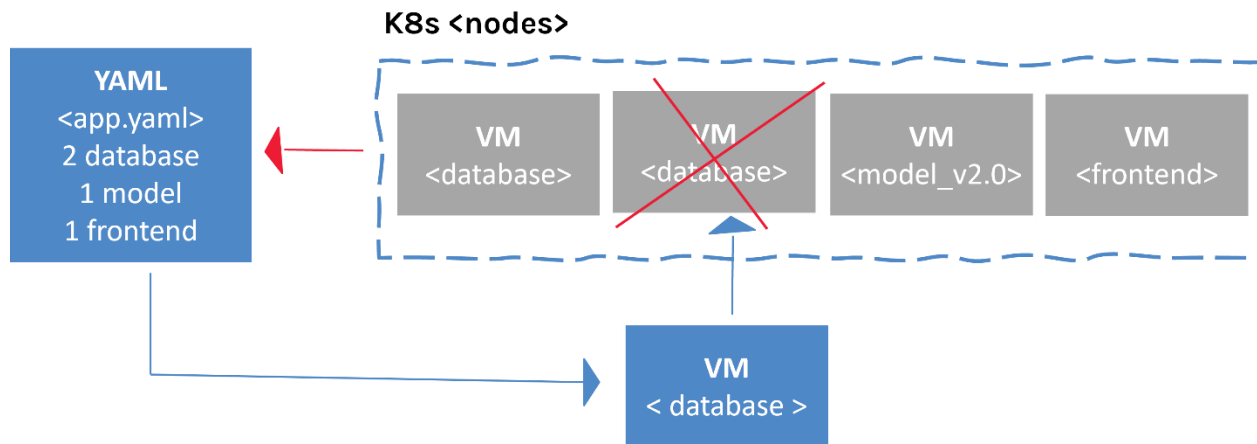
Imperative configuration are defined by the execution of a series of instructions, but not the other way around.



Velocity

Velocity is enabled by

- **Online self-healing systems:** k8s takes actions to ensure that the current state matches the desired state (as opposed to an operator enacting the repair)



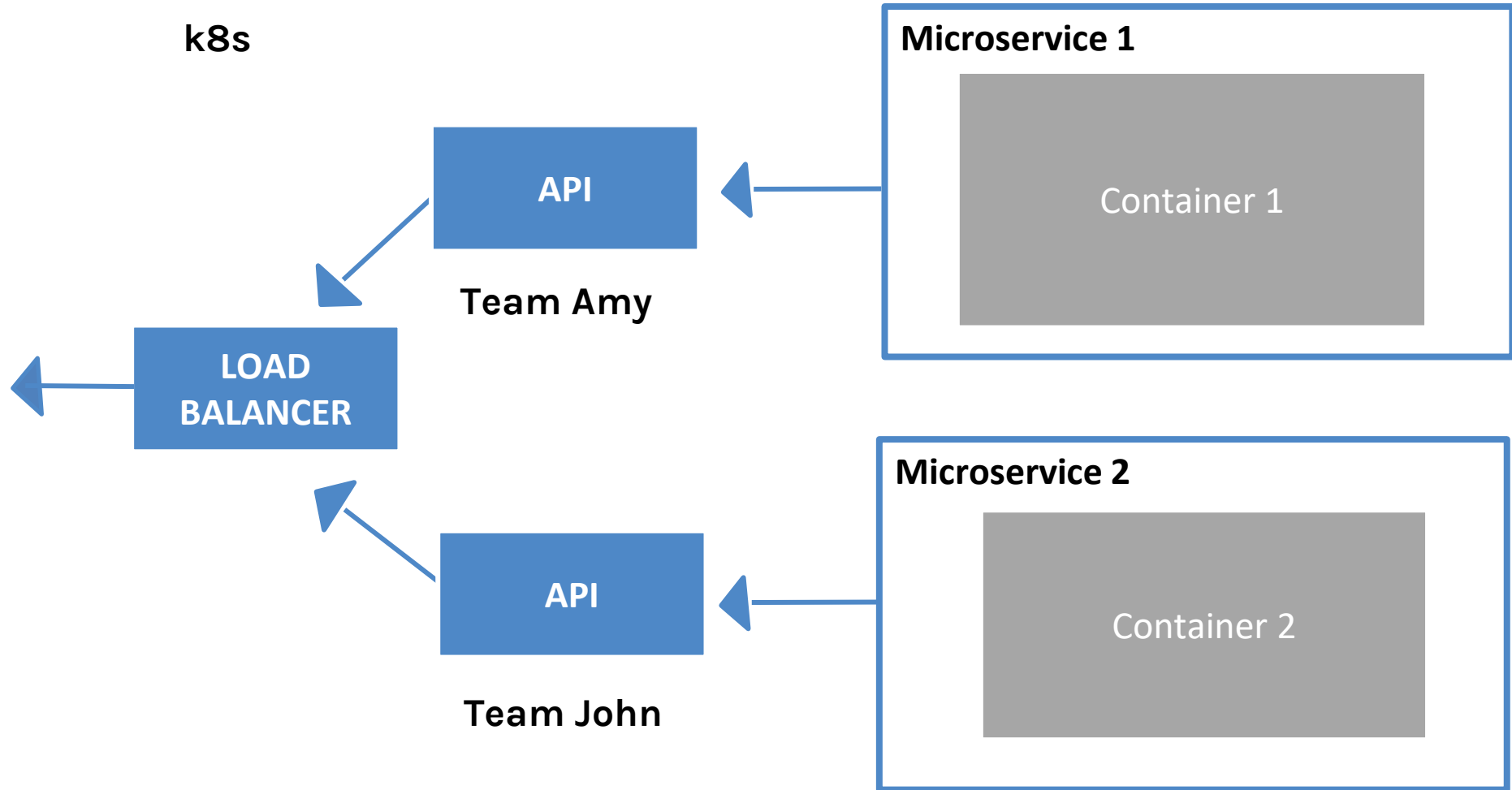
Scaling

Kubernetes provides numerous advantages to address scaling:

- **Decoupled architectures:** each component is separated from other components by defined APIs and service load balancers.
 - **Easy scaling for applications and clusters:** simply changing a number in a configuration file, k8s takes care of the rest (part of declarative).
 - **Scaling development teams with microservices:** small team is responsible for the design and delivery of a service that is consumed by other small teams.
-

Scaling

k8s



Abstracting Your Infrastructure

Kubernetes allows to build, deploy, and manage your application in a way that is portable across a wide variety of environments. The move to application-oriented container APIs like Kubernetes has two concrete benefits:

- **separation:** developers from specific machines
- **portability:** simply a matter of sending the declarative config to a new cluster



Efficiency

There are concrete economic benefit to the abstraction because tasks from multiple users can be packed tightly onto fewer machines:

- **Consume less energy** (ratio of the useful to the total amount)
- **Limit costs of running a server** (power usage, cooling requirements, datacenter space, and raw compute power)
- **Create quickly a developer's test environment** as a set of containers
- **Reduce cost of development instances in your stack**, liberating resources to develop others that were cost-prohibitive

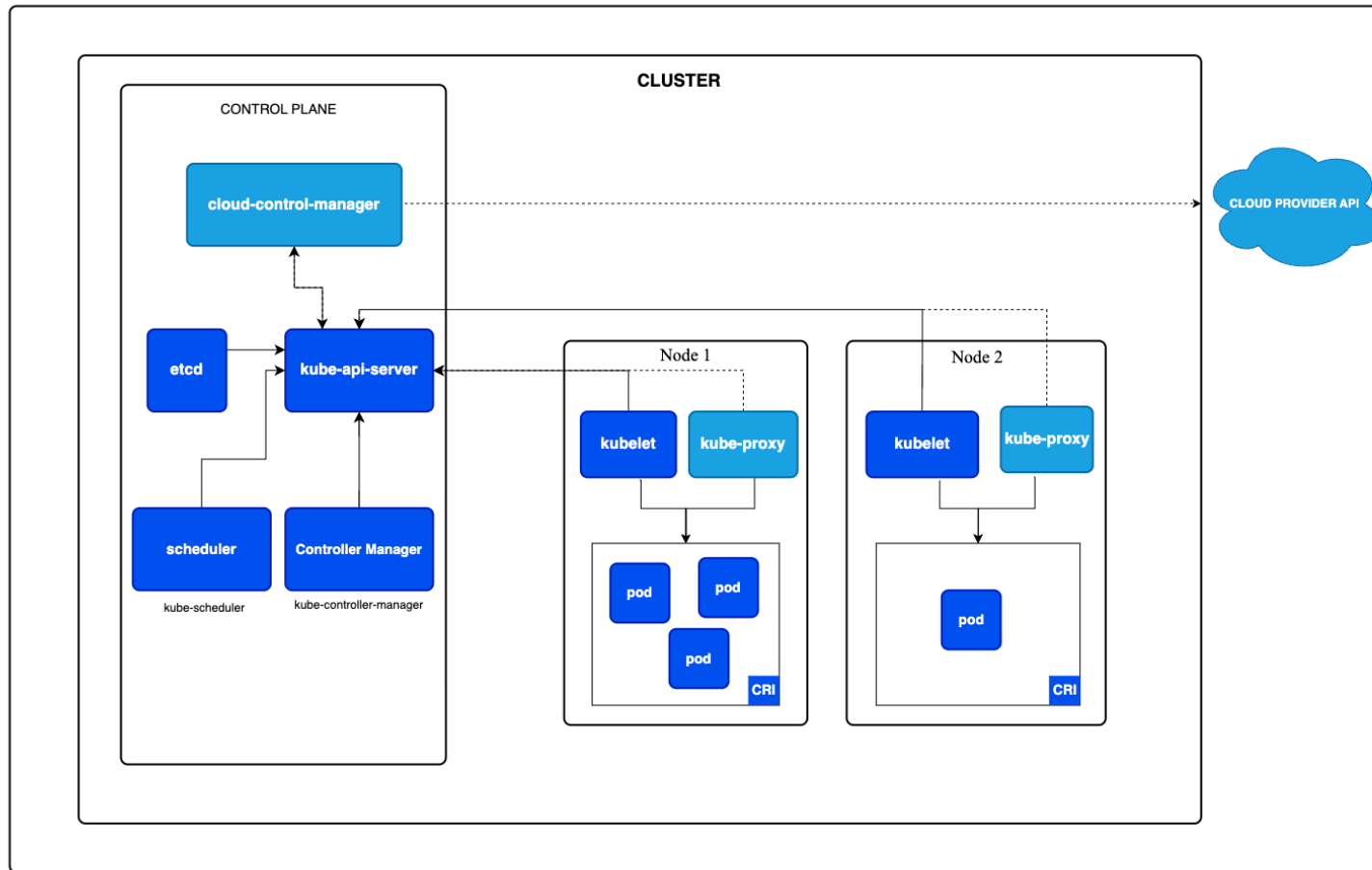
Features

- Designed to manage containerized applications.
 - Cluster consists of a set of nodes (physical or virtual machines) that run these applications.
 - The master node in a Kubernetes cluster is responsible for managing the cluster and coordinating tasks.
 - Machines in the cluster where applications are deployed and executed are called Worker nodes, also known as minions or nodes,
 - Each worker node runs a container runtime (such as Docker) and a Kubernetes agent called Kubelet, which communicates with the master node.
-

Features Contd.

- Smallest deployable unit in Kubernetes, representing a single instance of a running process is called a pod.
 - A Kubernetes service is an abstraction that defines a logical set of pods and a policy by which to access them.
 - Kubernetes supports multiple virtual clusters backed by the same physical cluster.
 - key-value pairs attached to objects in Kubernetes (such as pods, services, and deployments) to identify and organize them are called labels.
-

Cluster Architecture



Master Node

Master node main task is to manage the worker node(s) to run an application

The master node consists of:

- 1) **API server** contains various methods to directly access the Kubernetes
 - 2) **Scheduler** assigns to each worker node an application
 - 3) **Controller manager**
 - Keeps track of worker nodes
 - Handles node failures and replicates if needed
 - Provide endpoints to access the application from the outside world
 - 4) **Cloud controller** communicates with cloud provide regarding resources such as nodes and IP addresses
 - 5) **Etcd** works as backend for service discovery that stores the cluster's state and its configuration
-

Worker Node

A **worker node** consists of

- 1) **Container runtime** that pulls a specified Docker image and deploys it on a worker node
 - 2) **Kubelet** talks to the API server and manages containers on its node
 - 3) **Kube-proxy** load-balances network traffic between application components and the outside world
-

Worker Node

- Nodes are the worker machines
 - User applications are run on nodes.
 - Nodes can be physical or virtual machines.
 - Each node is managed by the control panel.
 - Nodes include pods.
 - Pods include one or more containers.
 - Containers share all the resources of the node and can communicate among themselves.
-

Control Panel - Kubernetes API Server

- Exposes Kubernetes API
 - It is the front end of the control panel.
 - All communications in the cluster utilize this API.
 - It accepts commands to view or change the state of the cluster.
 - Main implementation is kube-apiserver.
-

Control Panel - Etcd

- It is a distributed key-value store that contains all the cluster data.
 - When Kubernetes is asked to deploy application, the deployment configuration is stored in etcd.
 - Etcd defines the state of the Kubernetes cluster.
-

Control Panel - Scheduler

- Assigns newly created pods to nodes.
 - It determines where the workloads should run within the cluster.
 - It selects the most optimal node according to Kubernetes scheduling principles, configuration options and available resources.
-

Control Panel - Control Manager & cloud control manager

- Runs all the controller processes that monitor the cluster state
 - Cloud control manager runs controllers that interact with the underlying cloud providers.
 - These controllers link clusters into a cloud provider's API.
-

Nodes - kubelets

- Kubelet is the most important component of a worker node.
 - It communicates with the kube-apiserver to receive new and modified pod specifications.
 - Kubelet also reports to the control panel about pod' health and status.
 - In order to start a pod, the kubelet uses the container runtime.
 - Container runtime downloads images and running containers.
 - Docker is the best known runtime. Others Rodman and cri-o.
-

Nodes - kubernetes proxy

- A network proxy that runs on each node.
- It maintains network rules that allows communication to pods running on nodes.

Deploying a Kubernetes Cluster

- To deploy your cluster you must install Kubernetes.
- After installing minikube, use *start* to begin your session creating a virtual machine, *stop* to interrupt it, and *delete* to remove the VM. Below are the commands to execute these tasks:

```
$ minikube start  
$ minikube stop  
$ minikube delete
```

Deploying a Kubernetes Cluster

You can easily access the Kubernetes Client using the following command:

- to check your cluster status use:

```
$ kubectl get componentstatuses
```

- and should see output below:

NAME	STATUS	MESSAGE	ERROR
scheduler	Healthy	ok	
controller-manager	Healthy	ok	
etcd-0	Healthy	{"health": "true"}	